Codificação de Huffman: Uma aplicação de Árvores Binárias e Filas de Prioridade

O Propósito da Codificação Huffman

- **Proposta pelo Dr. David A. Huffman in 1952**
  - ***"Um método para a construção de códigos de redundância mínima"***
  - **Nosso exemplo: arquivos texto**

O Algoritmo Básico

- A codificação de Huffman é uma forma de codificação estatística
- Nem todos os caracteres ocorrem com a mesma frequencia
- Apesar disto, todos os caracteres ocupam, a princípio, o mesmo espaço
  - 1 char = 1 byte, seja ele e ou x

O Algoritmo Básico

- Será possível economizar, atribuindo códigos ligados à frequência dos caracteres?
- Os códigos não seriam mais os especificados pela tabela ASCII.
- O comprimento dos códigos variaria e seria mais curto para os caracteres mais frequentes.

O Algoritmo Básico

1. Faça o levantamento da frequência dos caracteres presentes no texto a ser compactado.
2. Ordene ou priorize os caracteres com base em sua ocorrência no texto.
3. Construa a Árvore de Huffman baseado na lista priorizada.
4. Percorra a árvore para determinar todos os possíveis códigos.
5. Processe o texto novamente e crie um novo arquivo usando os códigos de Huffman.

Construindo uma Árvore

Processe o texto original

- Considere o seguinte texto curto:

  Eerie eyes seen near lake.

- Conte quantas vezes cada um dos caracteres aparece no texto.

Construindo uma Árvore

Processe o texto original

Eerie eyes seen near lake.

- Quais são os caracteres presentes?

E e r i espaço

y s n a r l k .

Construindo uma Árvore

Processe o texto original

Eerie eyes seen near lake.

- Qual é a frequência de cada um dos caracteres no texto?

Char	Freq.	Char	Freq.	Char	Freq.
E	1	y	1	k	1
e	8	s	2	.	1
r	2	n	2		
i	1	a	2		
espaço	4	l	1		

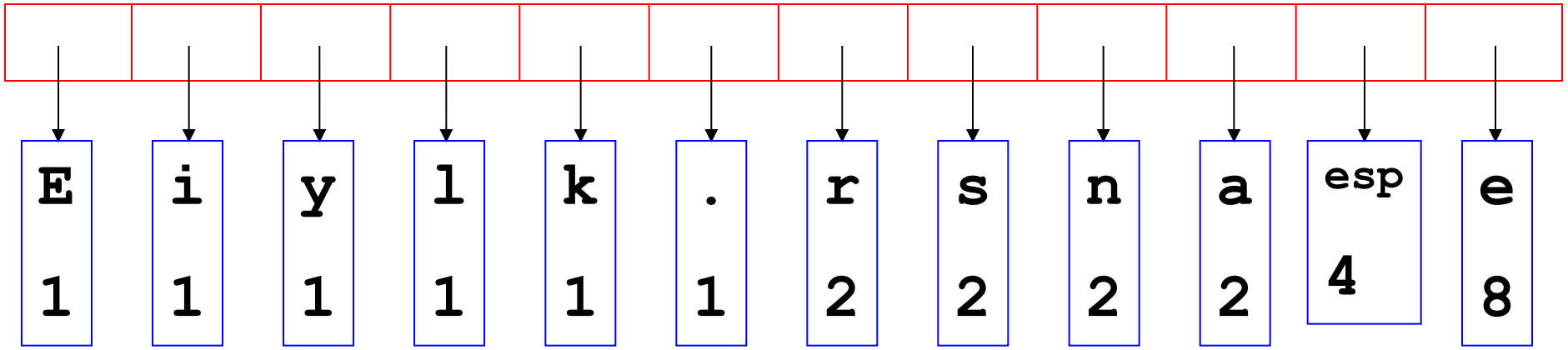
Construindo uma Árvore

Priorize os caracteres

- Crie nodos (nós) de árvore binária para cada um dos caracteres, contendo char/freq
- Coloque os nodos em uma fila de prioridade
  - Quanto menor a frequência, maior será a prioridade na fila

Construindo uma Árvore

- A fila após a inserção de todos os nodos

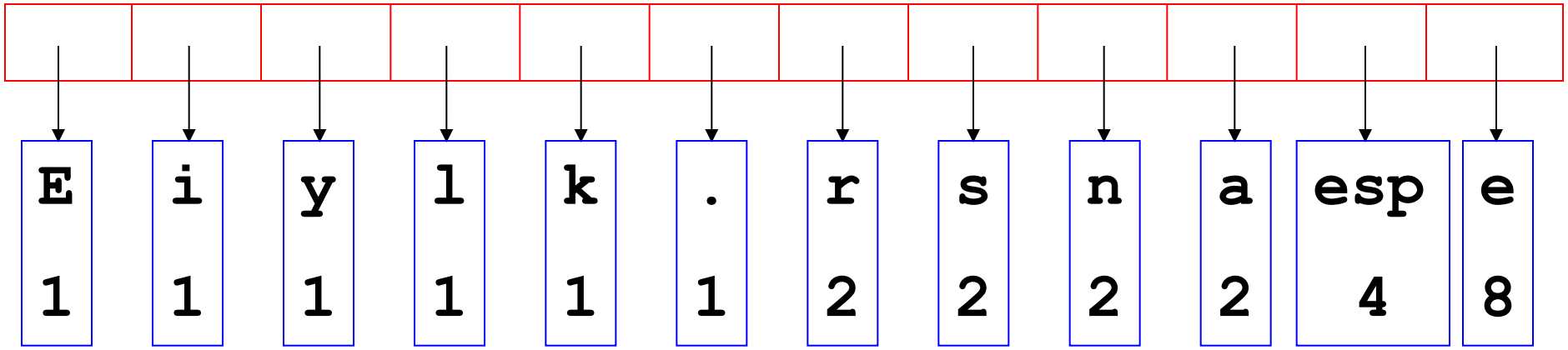


- Ponteiros nulos não foram mostrados

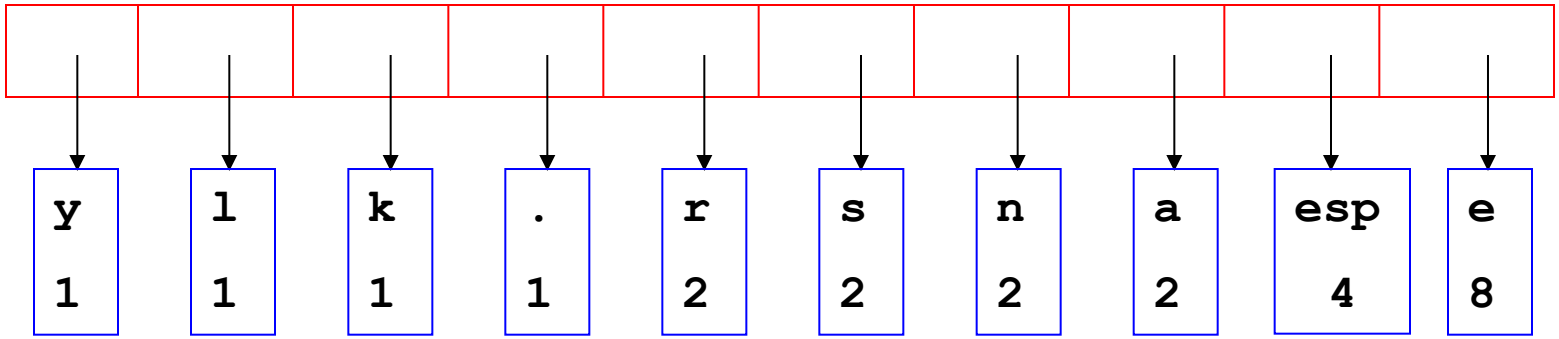
Construindo uma Árvore

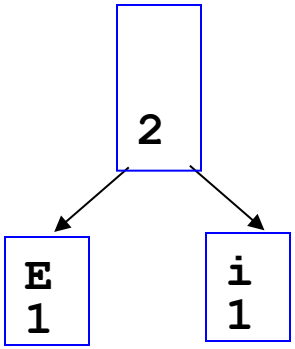
- Enquanto a fila contiver dois ou mais nodos
  - Crie um novo nodo
  - Desenfileire um nodo e o torne subárvore esquerda
  - Desenfileire um nodo e o torne subárvore direita
  - Torne a frequência do novo nodo igual à soma das frequências dos filhos esquerdo e direito
  - Enfileire o novo nodo

Construindo uma Árvore

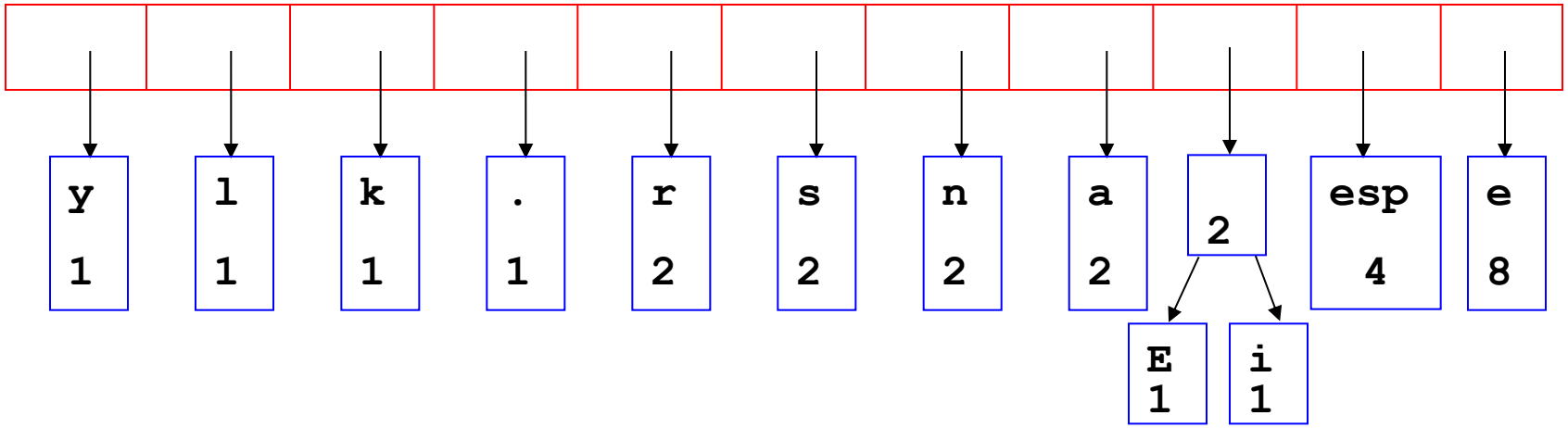


Construindo uma Árvore

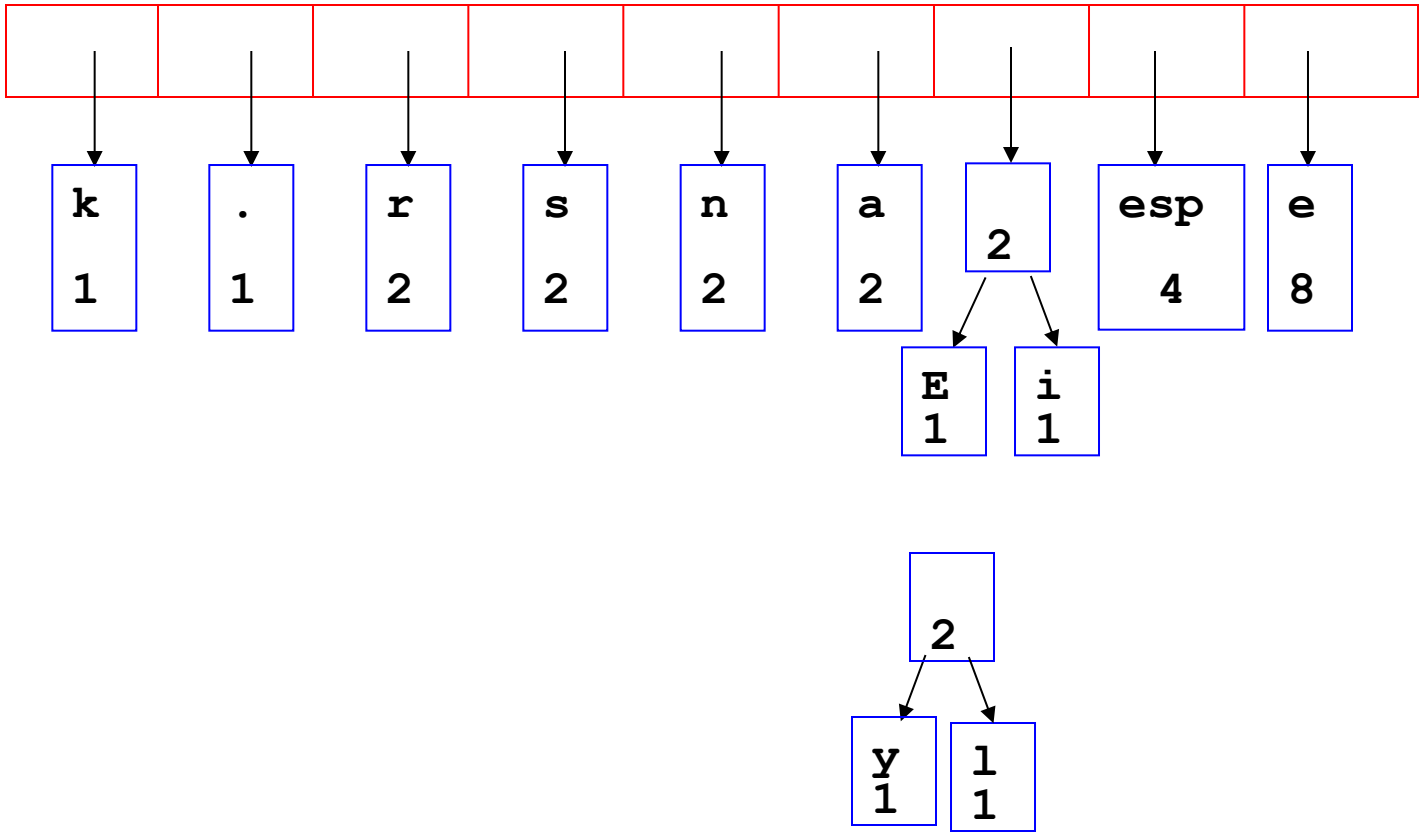




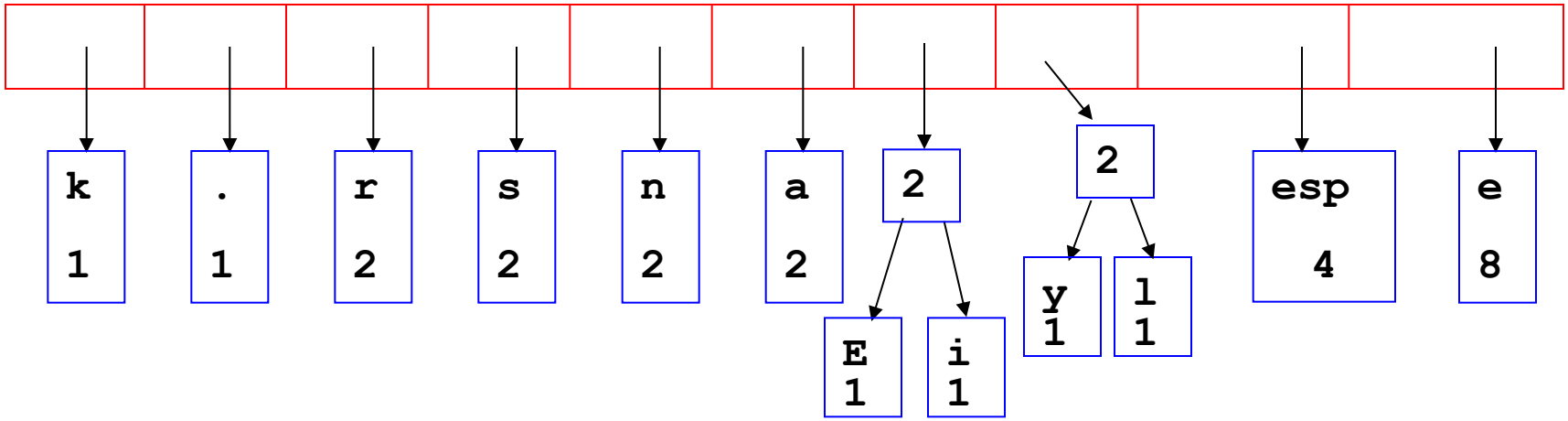
Construindo uma Árvore



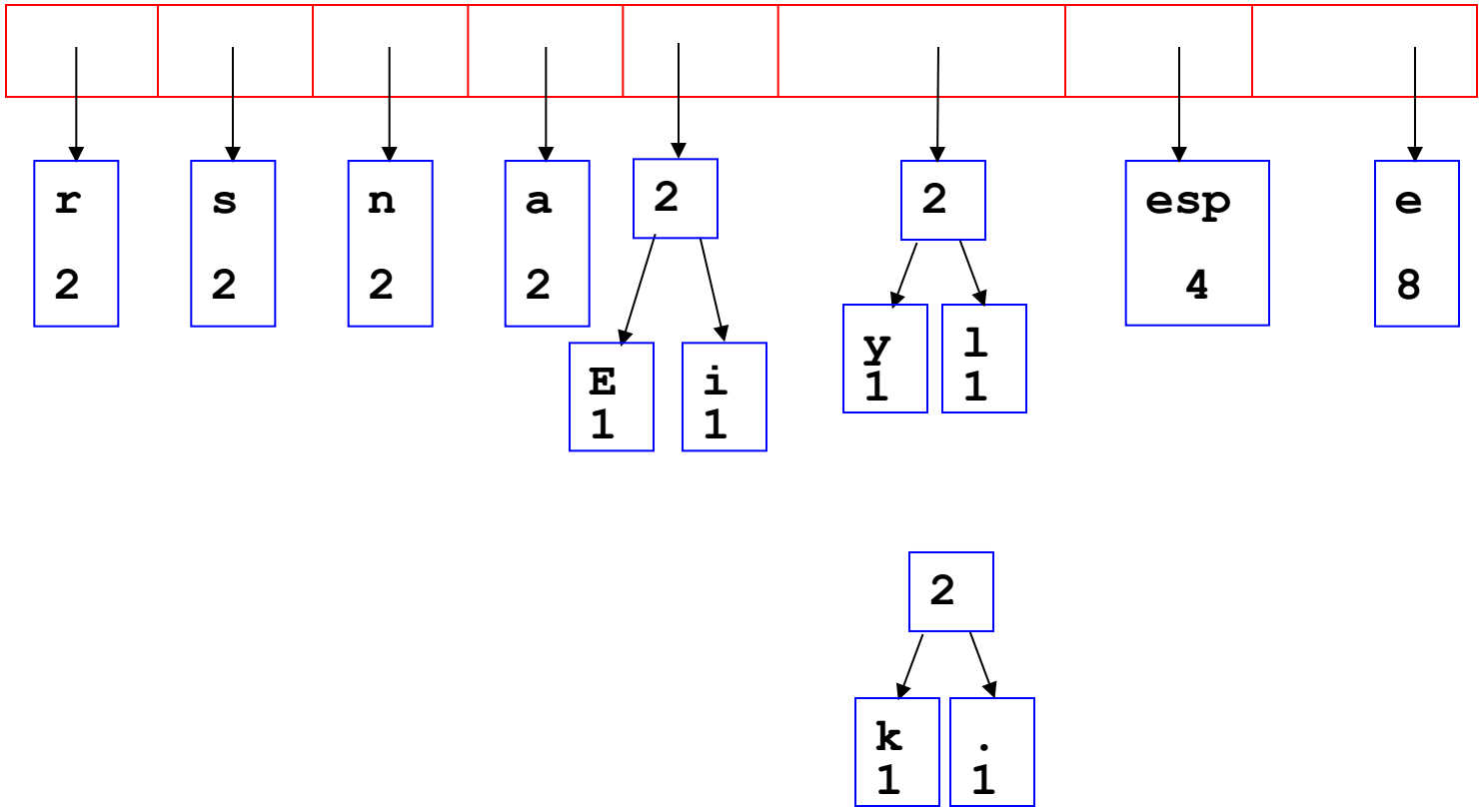
Construindo uma Árvore



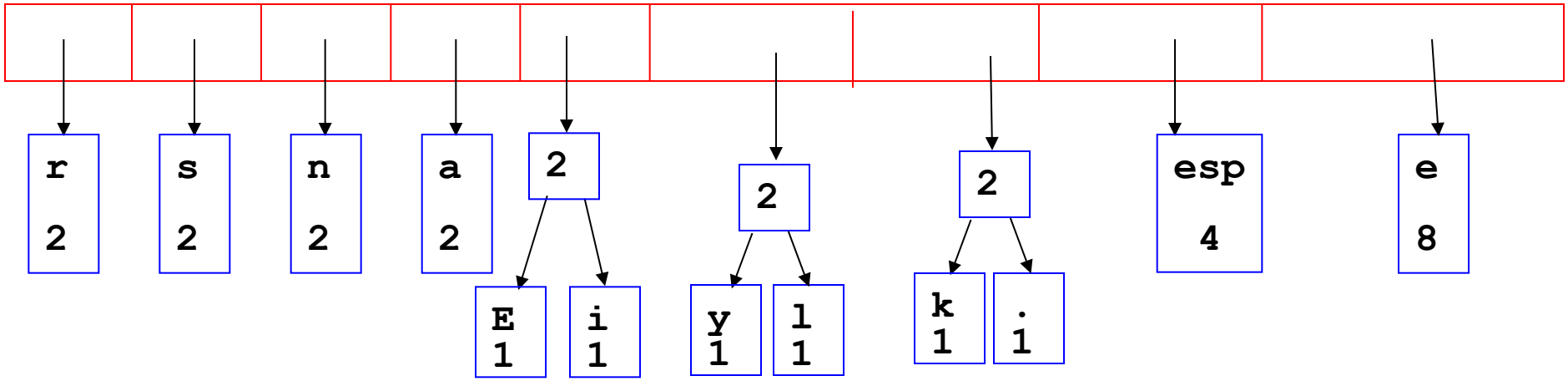
Construindo uma Árvore



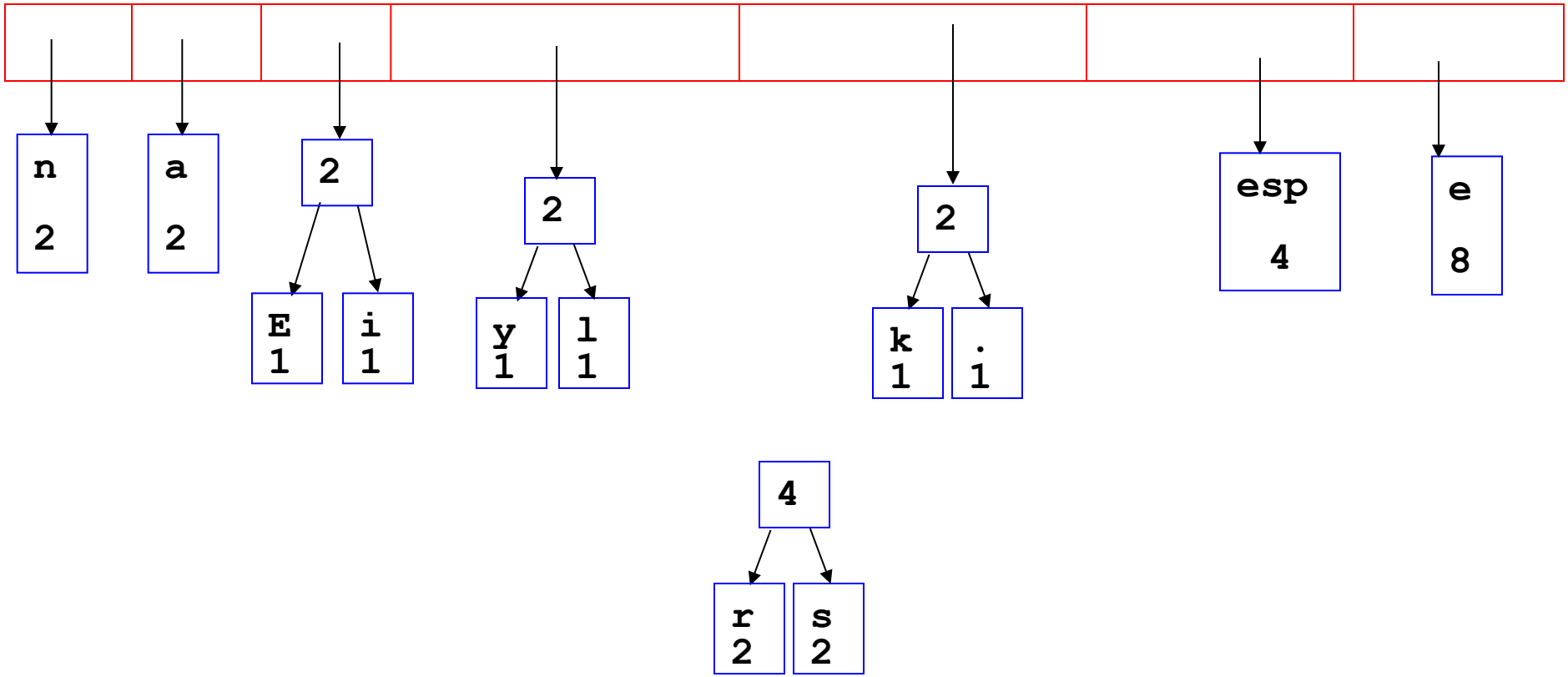
Construindo uma Árvore



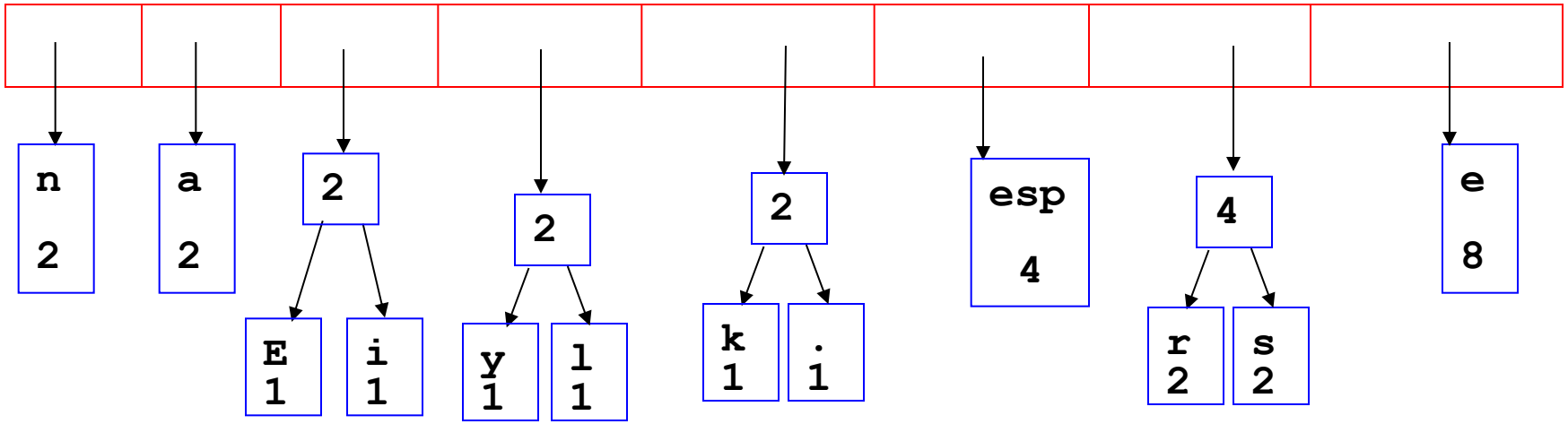
Construindo uma Árvore



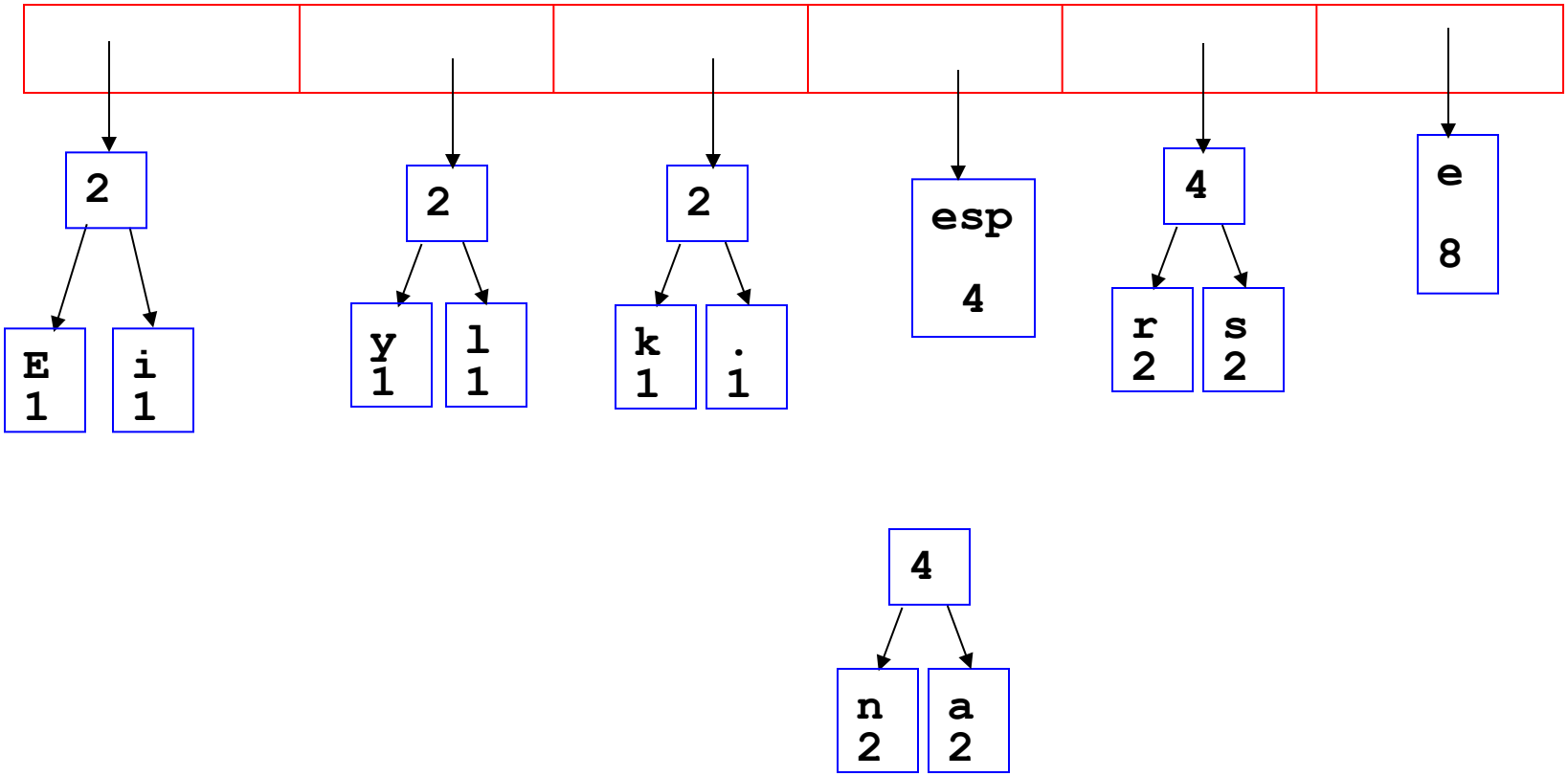
Construindo uma Árvore



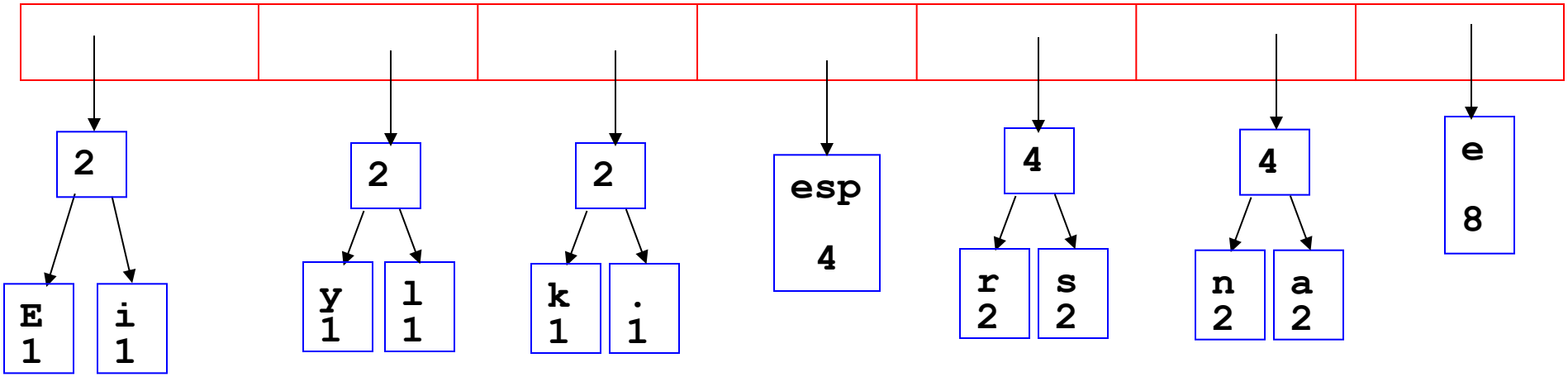
Construindo uma Árvore



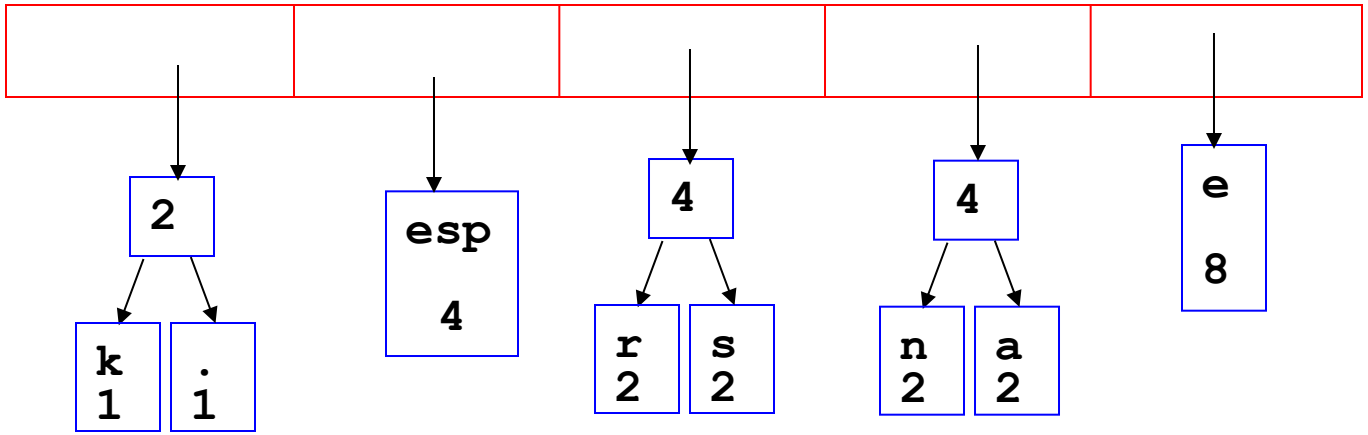
Construindo uma Árvore

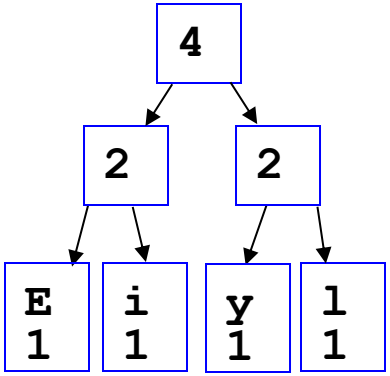


Construindo uma Árvore

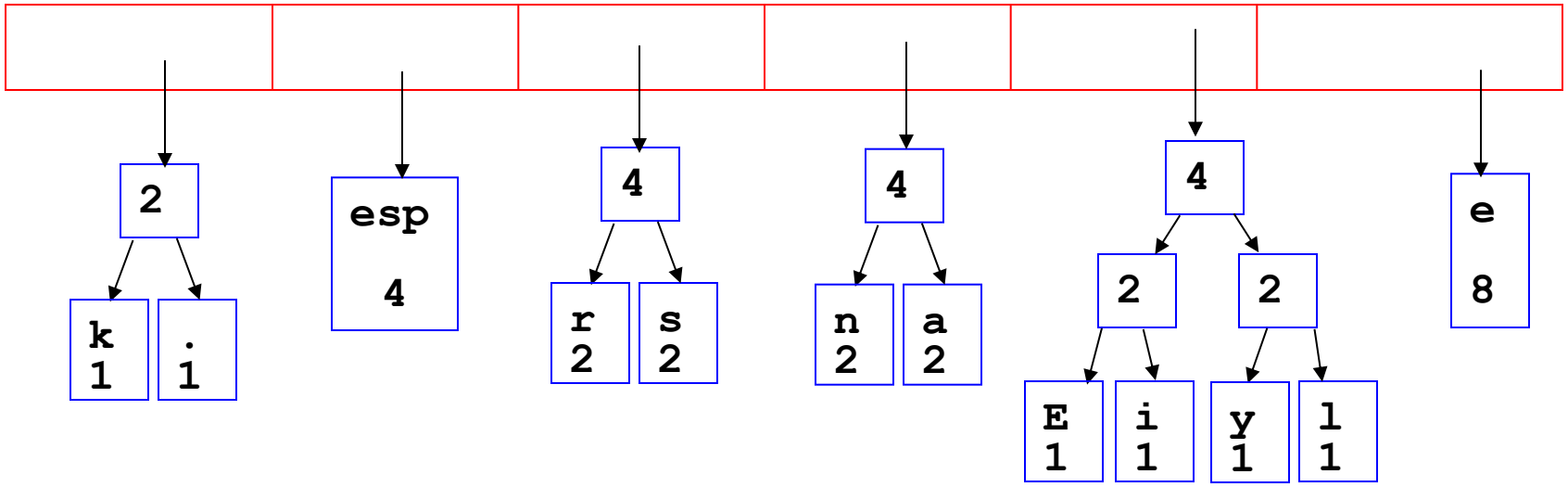


Construindo uma Árvore

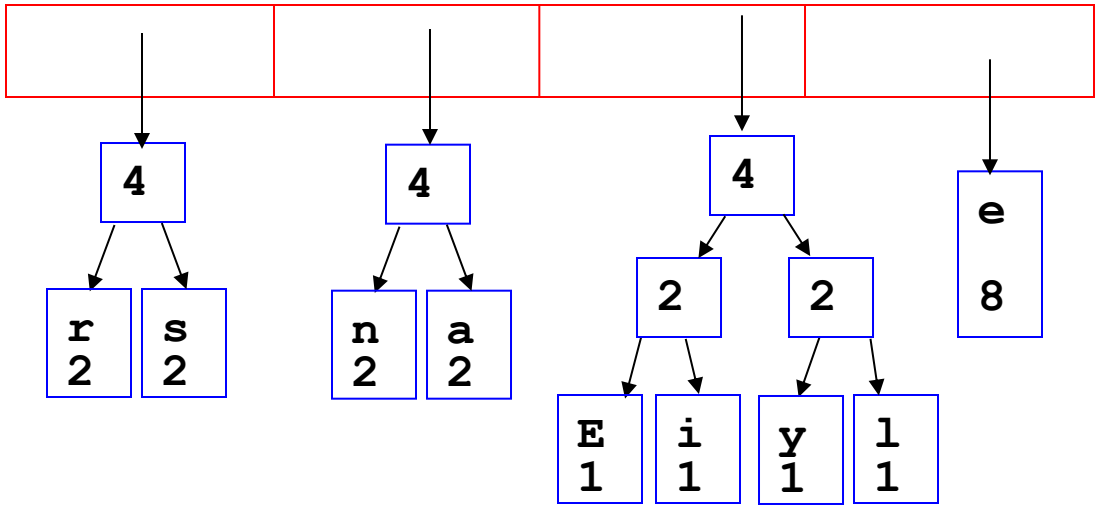


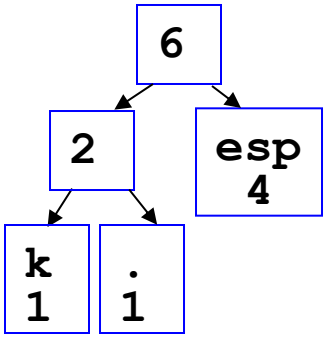


Construindo uma Árvore

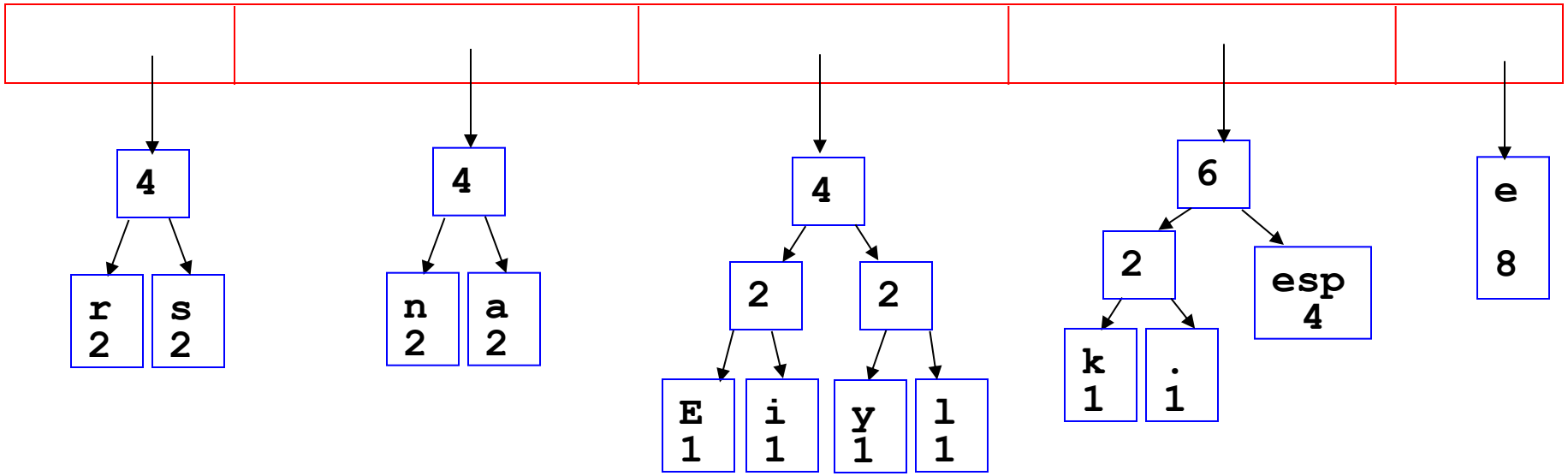


Construindo uma Árvore

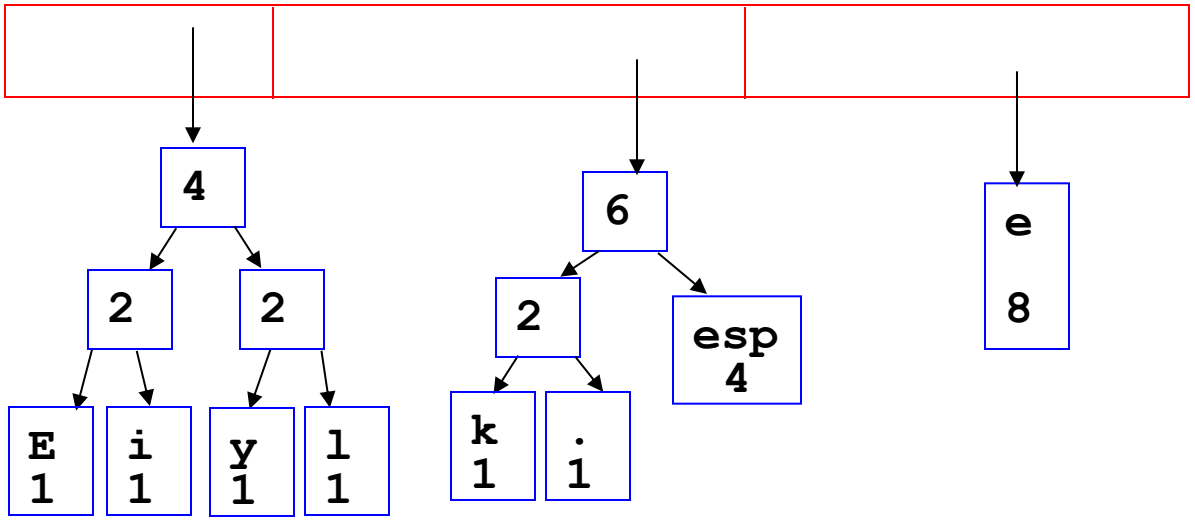


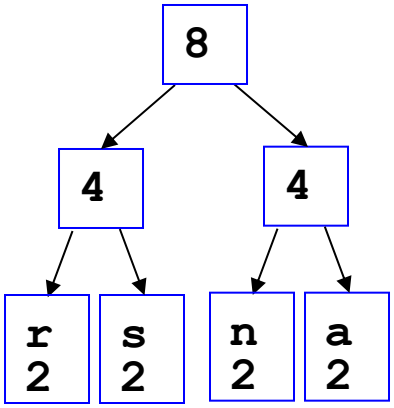


Construindo uma Árvore

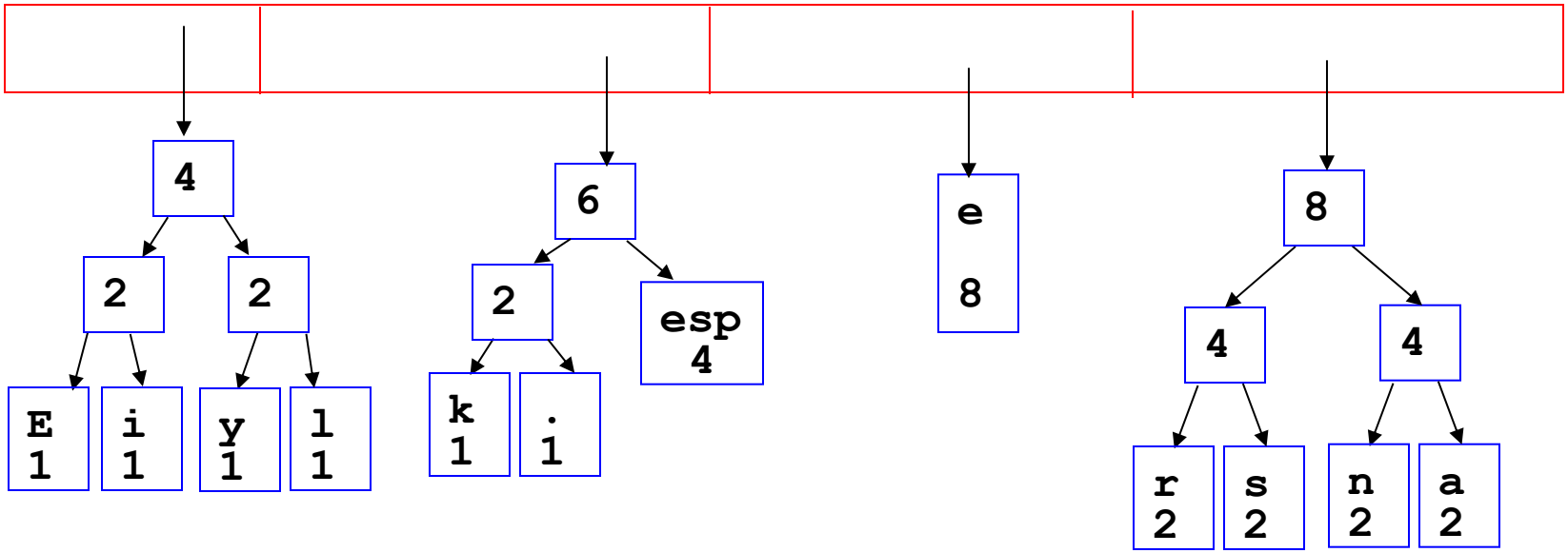


Construindo uma Árvore

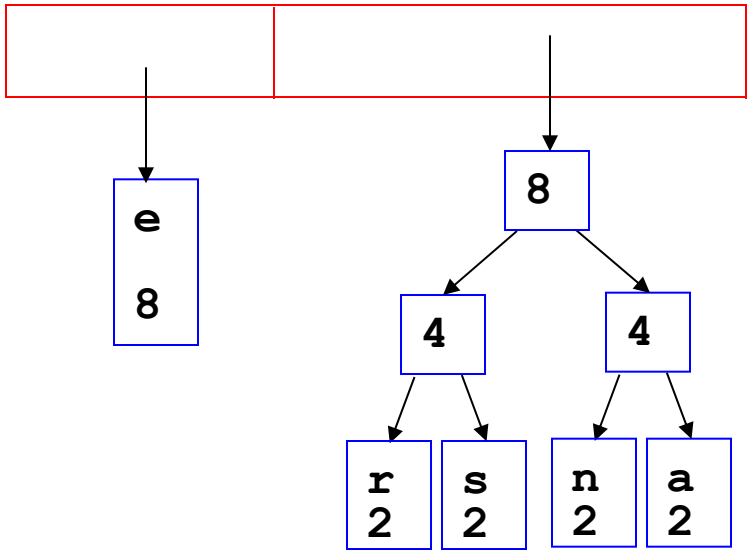


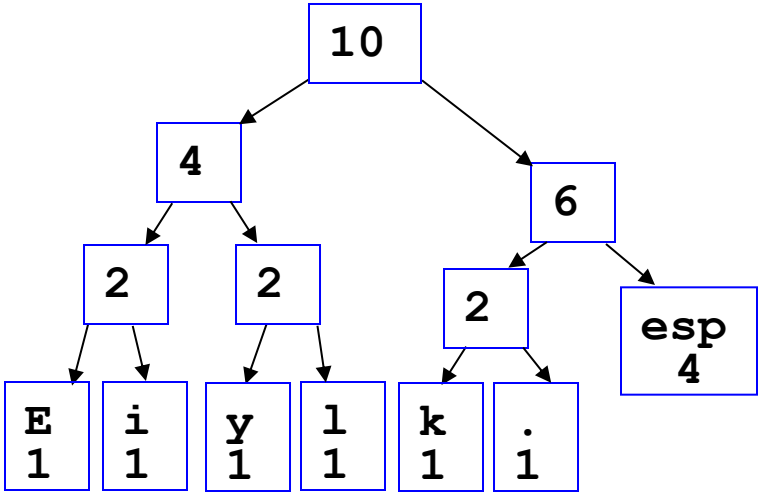


Construindo uma Árvore

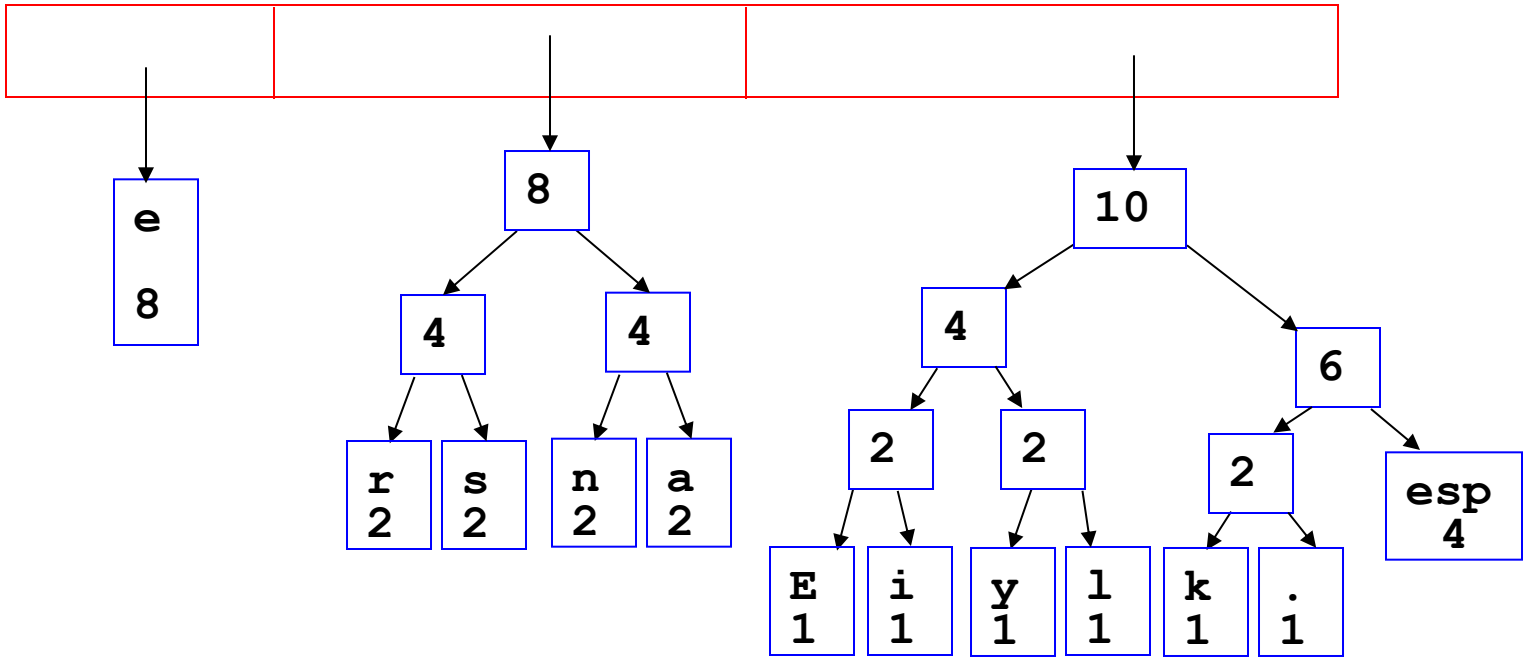


Construindo uma Árvore



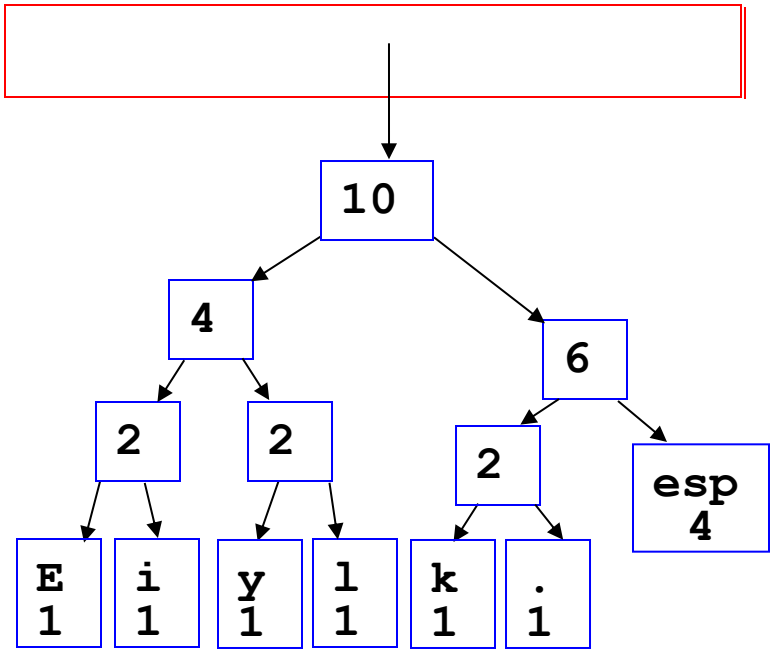


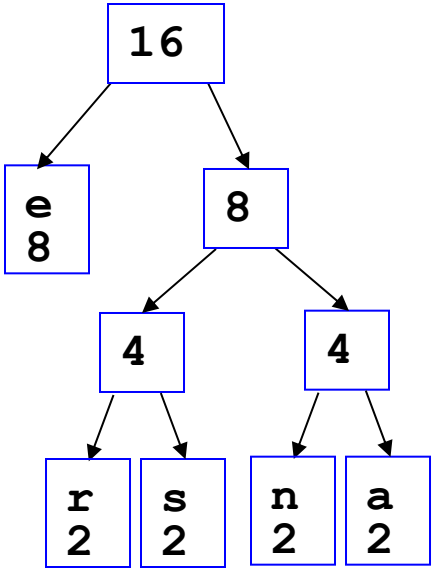
Construindo uma Árvore



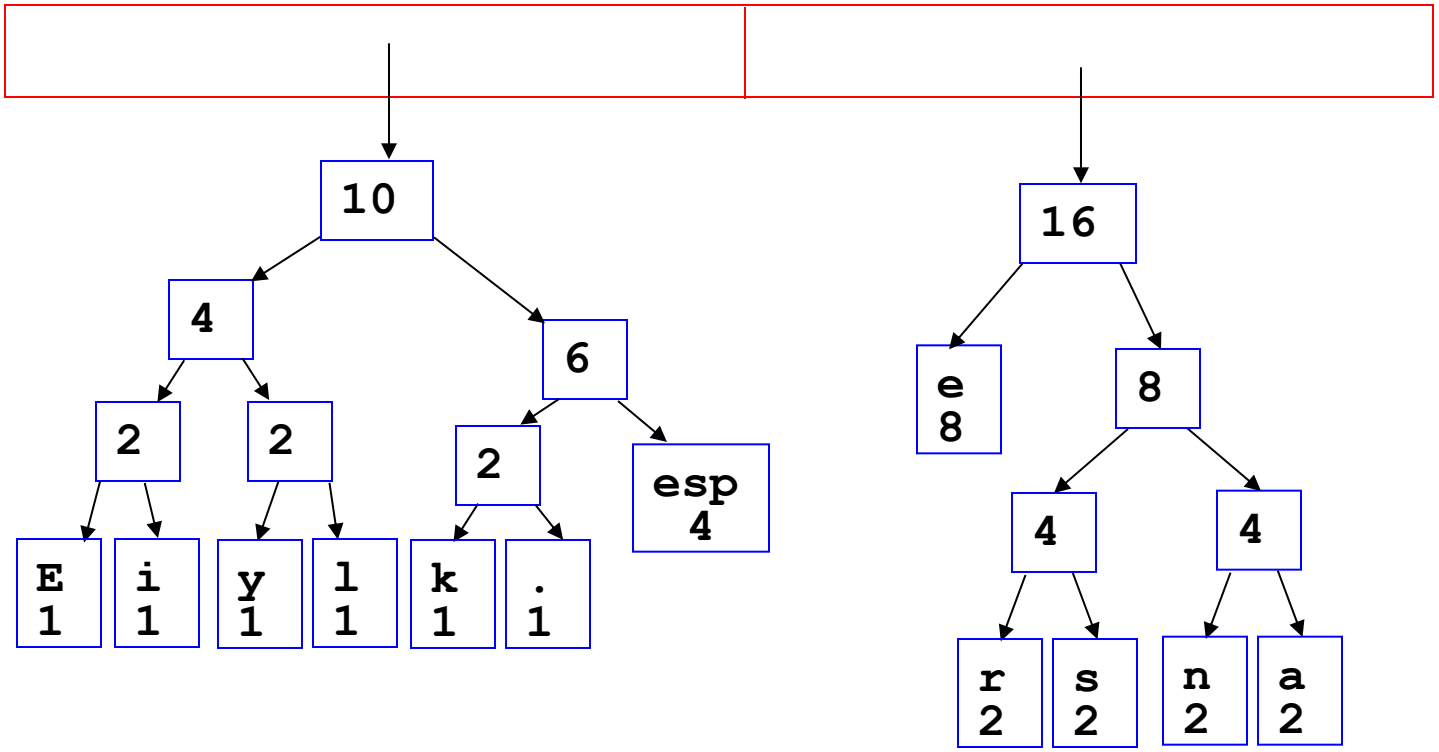
O que está acontecendo com os caracteres com baixa frequência?

Construindo uma Árvore

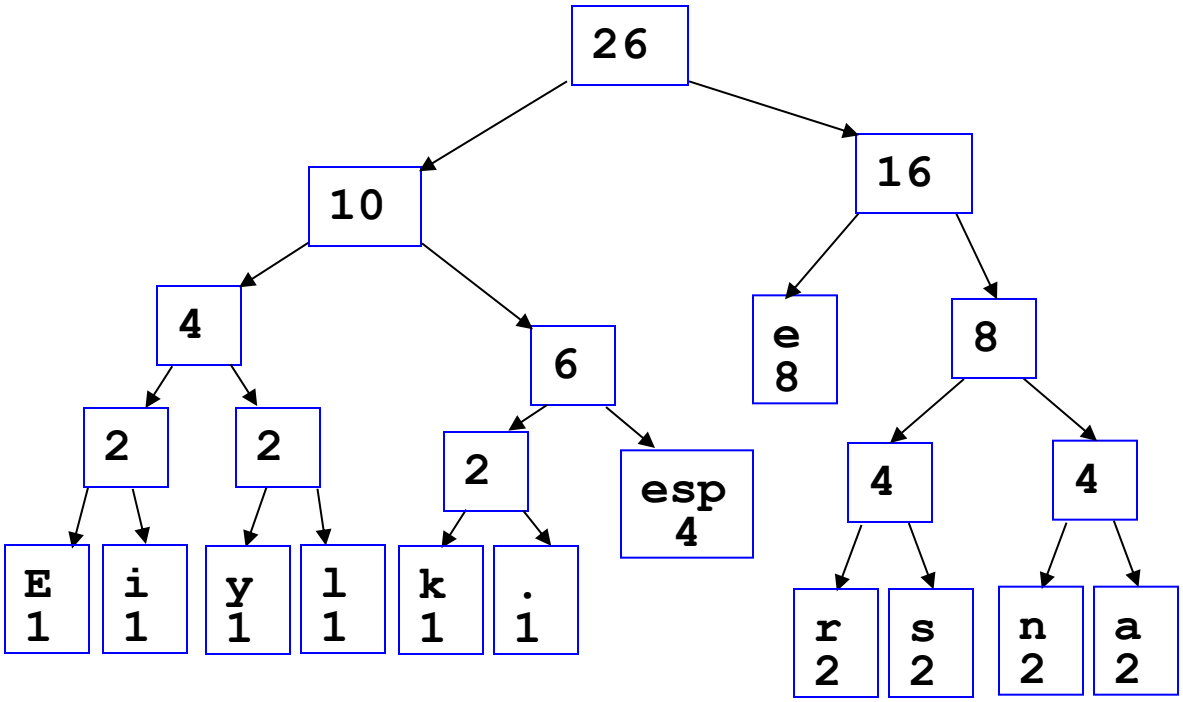




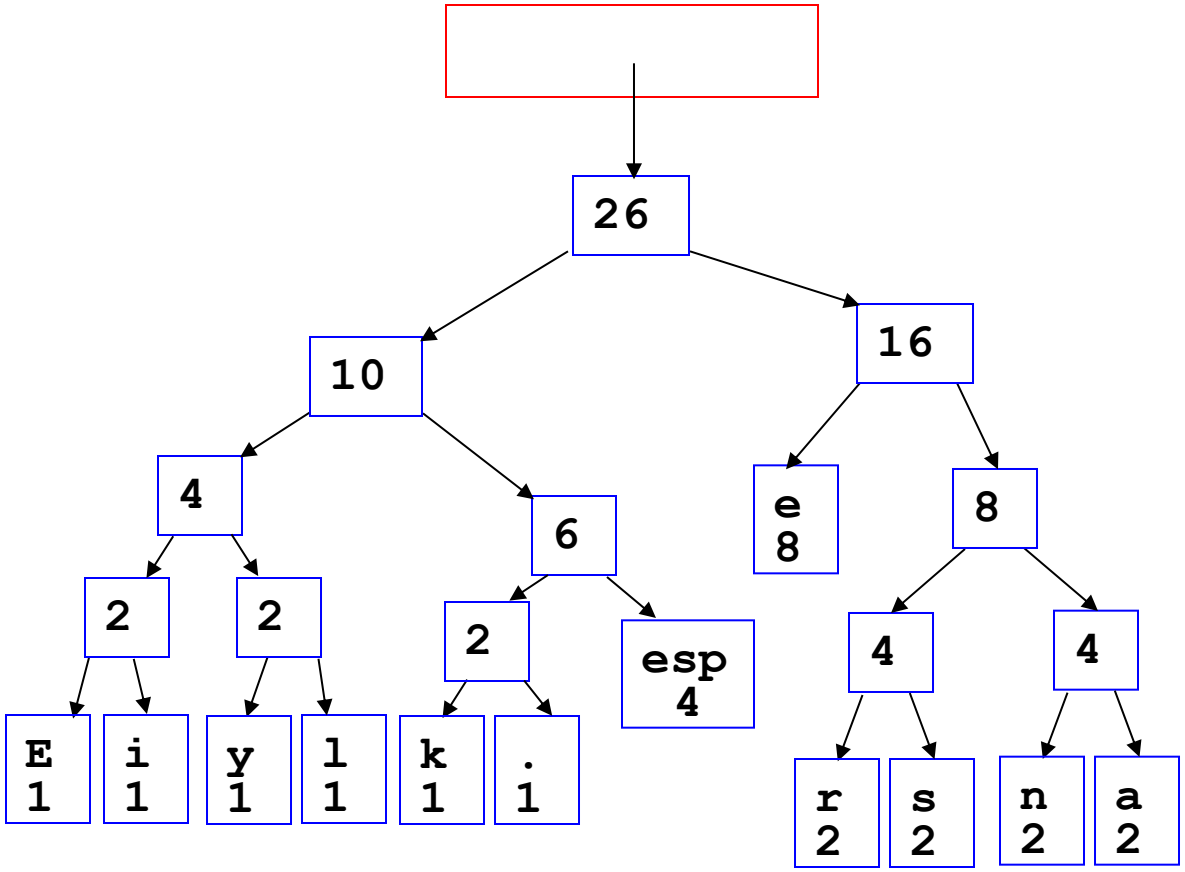
Construindo uma Árvore



Construindo uma Árvore



Construindo uma Árvore



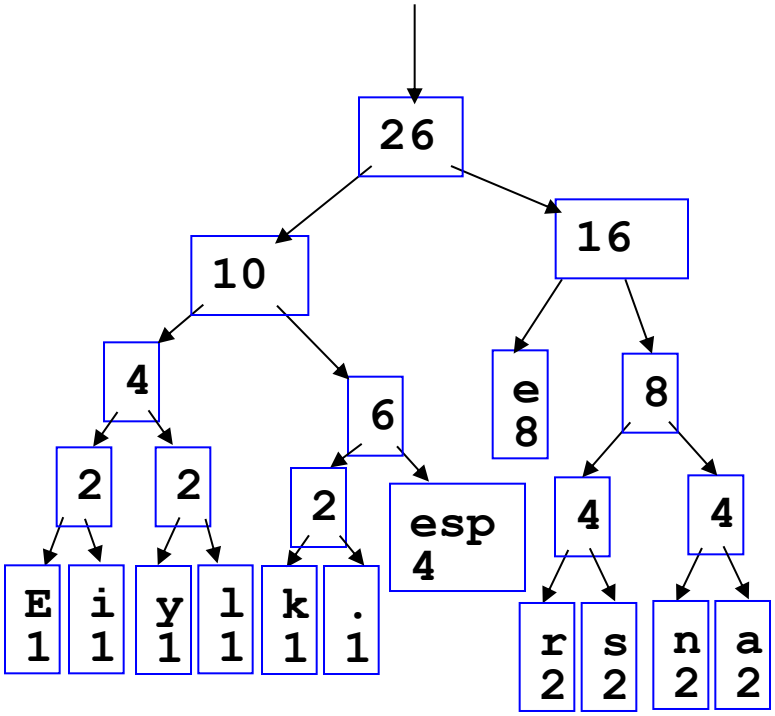
Depois de enfileirar este nodo há apenas um nodo na fila de prioridade

Construindo uma Árvore

Desenfileire este nodo.

Esta árvore contém os novos códigos de todos os caracteres.

A frequência contida no nodo raiz deve ser igual à quantidade total de caracteres no texto.



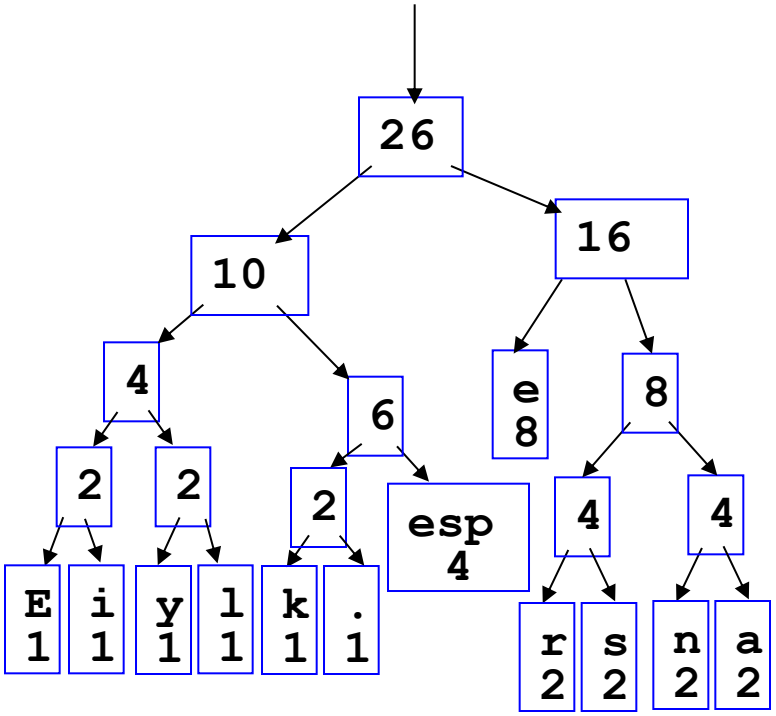
Eerie eyes seen near lake.

26 caracteres

Gerando os novos Códigos

Percorra a Árvore para ter os Códigos

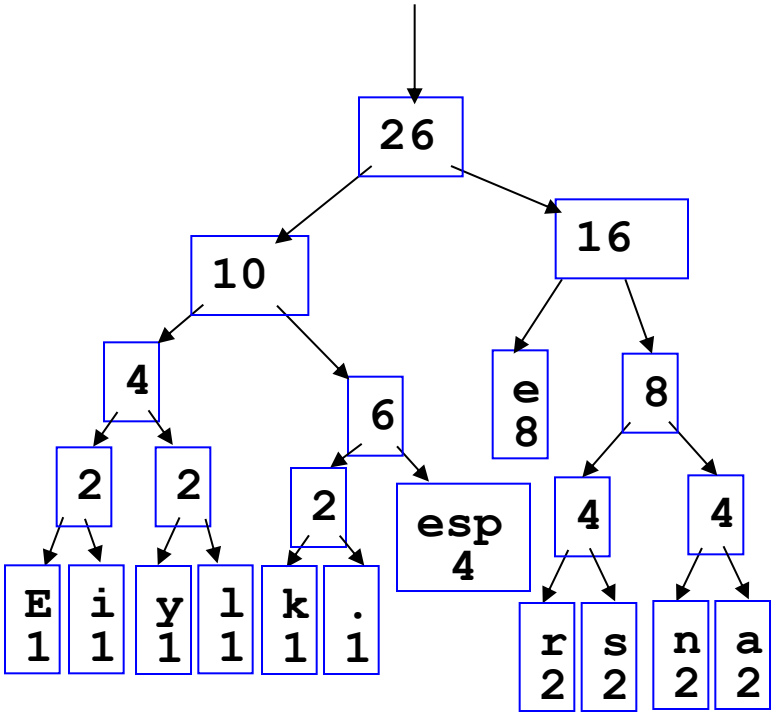
- Percorra a árvore para obter os novos códigos
- Acrescente 0 ao código, toda vez que for para a esquerda, ou 1, toda vez que for para a direita
- Um código está pronto quando uma folha for atingida



Gerando os novos Códigos

Percorra a Árvore para ter os Códigos

Char	Código
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
espaço	011
e	10
r	1100
s	1101
n	1110
a	1111



Compactando o Arquivo

- Reprocesse o texto e gere o arquivo compactado, usando os novos códigos

Eerie eyes seen near lake.

```
0000101100000110011
1000101011011010011
1110101111110001100
1111110100100101
```

- Porque não é necessário algo para separar os caracteres?

Char	Código
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
espaço	011
e	10
r	1100
s	1101
n	1110
a	1111

Compactando o Arquivo

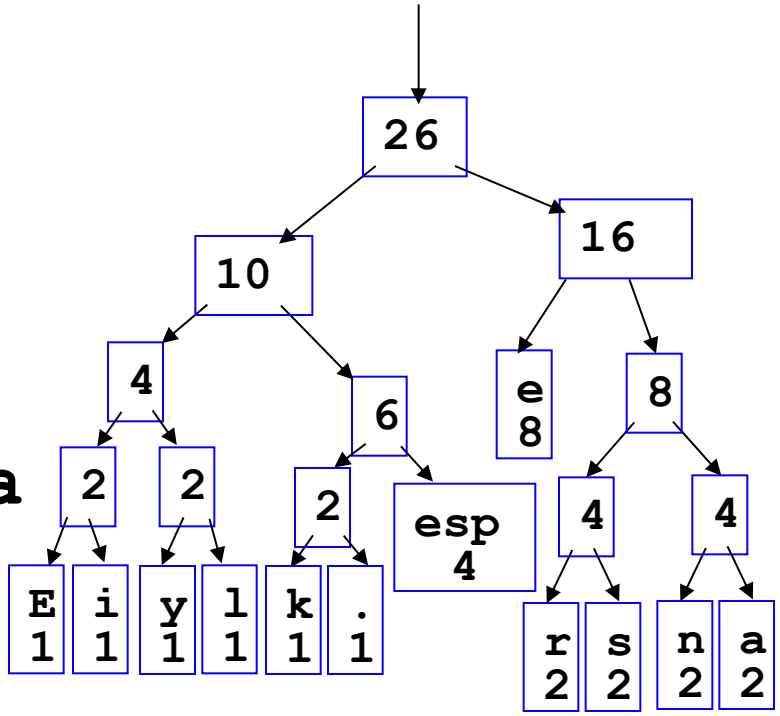
Resultados

- Conseguimos uma boa taxa de compactação?
- O texto compactado ocupou 73 bits
- ASCII ocuparia 8 * 26 = 208 bits
- Economizamos quase 65%

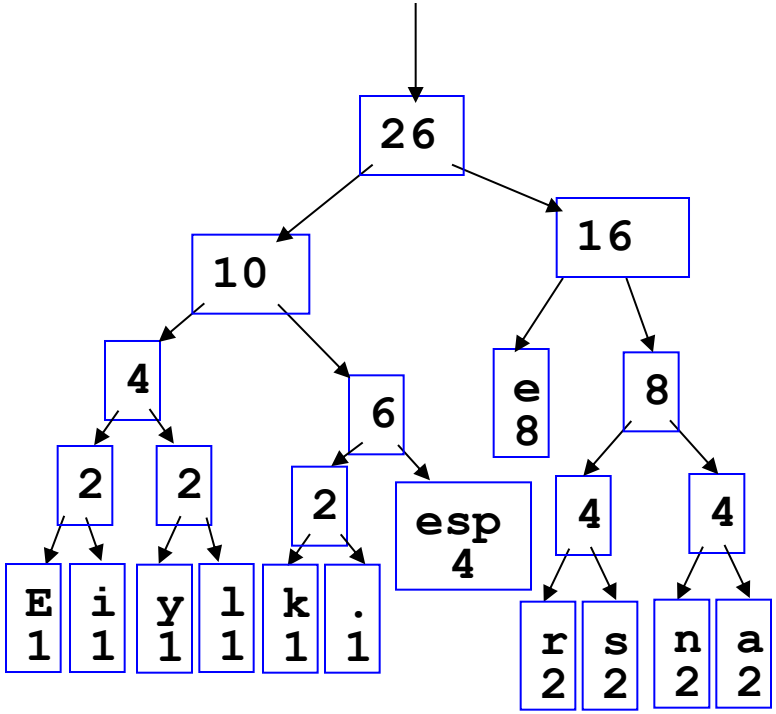
```
0000101100000110011
1000101011011010011
1110101111110001100
1111110100100101
```

Descompactando o Arquivo

- Como é que o descompactador sabe quais são os códigos dos caracteres?
- Tendo a árvore, o descompactador processa os bits do arquivo a ser descompactado
- 0 ⇒ vai p/ esquerda
- 1 ⇒ vai p/ direita



Descompactando o Arquivo




```
000010110000011001 1
100010101101101001 1
111010111111000110 0
111111010010010 1
```

- 0 ⇒ vai p/ esquerda
- 1 ⇒ vai p/ direita

Resumo

- A Codificação de Huffman é uma técnica usada para compactar arquivos.
- Ela usa codificação estatística:
  - caracteres mais frequentes têm códigos mais curtos.
- A descompactação, necessariamente, precisa da árvore e, para tê-la, pode ser necessário sacrificar a taxa de compactação.
- Várias estruturas de dados são usadas.